



UNIVERSIDAD PRIVADA DE TACNA

Proyecto: Sistema Verificador de Cumplimiento OWASP

Curso: Calidad Y Pruebas De Software

Docente: Mag. Ing. Patrick Cuadros Quiroga

Integrantes:

Concha Llaca, Gerardo Alejandro

(2017057849)

Andia Navarro, Diego Fabrizio

(2022073906)

Tacna – Perú

2026



CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
07	Andia Navarro, Diego Fabrizio Concha Llaca, Gerardo Alejandro	Mag. Ing. Patrick Cuadros Quiroga	Mag. Ing. Patrick Cuadros Quiroga	27/06/26	

Sistema Verificador de Cumplimiento OWASP

***Documento de Arquitectura de Software
Versión {1.0}***



ÍNDICE GENERAL

1.	Introducción	4
1.1	Propósito	4
1.2	Alcance	4
1.3	Definición, siglas y abreviaturas	4
1.4	Referencias	4
1.5	Visión General	5
2.	Representación Arquitectónica	5
2.1	Escenarios	5
2.2	Vista Lógica	6
2.3	Vista del Proceso	6
2.4	Vista del desarrollo	6
2.5	Vista Física	6
3.	Objetivos y limitaciones arquitectónicas	7
3.1	Seguridad	7
3.2	Escalabilidad	7
3.3	Tiempos de Respuesta Rápidos	7
3.4	Optimización de Carga	7
4.	Análisis de Requerimientos	8
4.1	Requerimientos funcionales	7
4.2	Requerimientos no funcionales	7
5.	Vistas de Caso de Uso	9
6.	Vista Lógica	17
6.1	Diagrama Contextual	17
7.	Vista de Procesos	18
7.1	Diagrama de Proceso Actual	18
7.2	Diagrama de Proceso Propuesto	18
8.	Vista de Despliegue	19
8.1	Diagrama de Contenedor	19
9.	Vista de Implementación	20
9.1	Diagrama de Componentes	20
10.	Vista de Datos	22
10.1	Diagrama Base de Datos	22
11.	Calidad	24
11.1	Escenario de Seguridad	25



11.2	Escenario de Escalabilidad	26
11.3	Escenario de Tiempos de Respuesta Rápidos	27
11.4	Optimización de Carga	28



1. Introducción

1.1 Propósito

El propósito de este documento es definir y comunicar de manera integral la arquitectura de software del **Ecosistema Verificador de Cumplimiento OWASP**, detallando su estructura en capas, los flujos de procesos, las integraciones con servicios externos (como la API de GitHub para reporte de incidencias), y las decisiones de diseño que garantizan el cumplimiento de los requisitos funcionales y no funcionales, con el fin de guiar el desarrollo, asegurar la escalabilidad, facilitar el mantenimiento y servir como base de referencia para todos los stakeholders involucrados en el proyecto.

Este documento detalla las vistas arquitectónicas del sistema (lógica, de procesos, de desarrollo y física), con el objetivo de:

- Facilitar la comprensión técnica del sistema.
- Guiar el desarrollo y mantenimiento modular del código.
- Garantizar la escalabilidad para futuras adiciones de firmas de vulnerabilidad.
- Establecer estándares para la integración con servicios externos (GitHub y APIs REST).

1.2 Alcance

Este sistema (Web, CLI y Extensión de IDE) estará dirigido en su versión piloto a la consultora Securify Software Solutions S.A.C. e ingenieros asociados. Incluirá funcionalidades de escaneo estático de código fuente, análisis de dependencias vulnerables (CVEs), verificación de cabeceras HTTP de URLs en la nube y persistencia en SQLite3/MySQL. El sistema estará disponible para acceso remoto a través de APIs de red y será compatible con el editor VS Code y los principales navegadores web.



Término	Definición
SAST	Static Application Security Testing: Pruebas de seguridad estáticas que analizan el código fuente sin ejecutarlo.
OWASP	Open Web Application Security Project: Estándar y consorcio mundial que define los 10 riesgos de seguridad web más críticos.
CVE	Common Vulnerabilities and Exposures: Base de datos pública y estandarizada que recopila fallos y vulnerabilidades de dependencias.
API REST	Interfaz basada en protocolo HTTP utilizada para conectar la extensión del IDE VS Code y el CLI con el servidor backend.
SSO / Cookies	Mecanismo de autenticación web unificada y persistencia de sesión en memoria para administradores.

1.3 Definición, Siglas y Abreviaturas

1.4 Referencias

Documentos:

- Especificación de Requerimientos de Software (SRS - FD03).
- Guía de Controles y Mitigaciones OWASP Top 10 (2021).

Tecnologías:

- Documentación oficial de Python 3.11 y FastAPI.
- API de Extensiones de Microsoft VS Code.
- Motores de Base de Datos MySQL y SQLite3.

Autenticación y Autorización:

- Tokens de acceso API (**X-API-Key**).
- Cookies seguras de sesión de administración en memoria (con expiración automática de 6h).
-



1.5 Visión General

El sistema sigue una arquitectura modular en capas con:

- **Presentación (Frontend):** Pistas y subrayados diagnósticos interactivos en el editor de código, Dashboard premium con estilos glassmorphism (Webview en VS Code) e interfaz web administrativa con plantillas Jinja2.
- **Backend:** API REST rápida implementada con Python 3.11, FastAPI y routers modulares ([analysis.py](#), [dashboard.py](#), [reports.py](#)).
- **Base de Datos:** Almacenamiento relacional de logs de red y auditorías mediante base de datos SQLite3 (local) y MySQL (producción desplegada en servidor Debian en la nube).
- **Servicios Externos:**
 - **GitHub API:** Integración para sincronización e inserción automática de issues/tickets de seguridad.



2. Representación Arquitectónica

2.1 Escenarios

Tabla 1

Escenario: Escanear Objetivos

Escenario de caso de uso: Escanear objetivos (código, URL y repositorios de GitHub)	
Tipo	Obligatorio
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Usuario, API de GitHub (Actor Secundario)
Descripción	Permite al usuario realizar auditorías de seguridad sobre diferentes objetivos (archivos de código locales, texto pegado, URLs públicas de producción o repositorios remotos de GitHub) para verificar el cumplimiento de las directrices OWASP Top 10.
Precondiciones	El usuario debe encontrarse en el formulario de análisis del portal web o enviar una petición HTTP REST.
Postcondiciones	Se genera un reporte detallado con los hallazgos de seguridad detectados, se calcula el puntaje final de cumplimiento (de 0 a 100) y se guarda el historial del análisis en la base de datos (MySQL/SQLite).
Narrativa del Caso de Uso (Flujo Principal - Código Local por Archivo)	
Acción del Actor	Respuesta del Sistema
1. El usuario selecciona la opción "Archivo" en el formulario de análisis.	2. El sistema habilita el cargador de archivos locales del navegador.
3. El usuario selecciona su archivo de código local (.py, .js, etc.) y hace clic en "Analizar".	4. El sistema lee el contenido del archivo, valida que no esté vacío y ejecuta el motor estático aplicando expresiones regulares y buscando vulnerabilidades.
	5. El sistema calcula la puntuación final de cumplimiento (restando penalidades de 30, 15 y 5 puntos según severidad).
	- El sistema persiste el reporte de escaneo en la base de datos y redirige a la vista detallada (/reports/{scan_id}).
Flujo Alternativo - FA1 (Cargar código mediante Texto Directo)	
Acción del Actor	Respuesta del Sistema
1. El usuario selecciona la opción "Texto" en el formulario.	2. El sistema oculta el cargador de archivos y habilita una caja de texto grande para entrada directa.
3. El usuario pega el fragmento de código en la caja de texto y hace clic en "Analizar".	4. El sistema lee el texto directo, valida que no esté vacío y continúa con el paso 4 del **Flujo Principal** .
Flujo Alternativo - FA2 (Analizar URL Web Pública)	
Acción del Actor	Respuesta del Sistema



1. El usuario selecciona la opción "URL" en el formulario de análisis.	2. El sistema habilita el campo de texto para ingresar direcciones web.
3. El usuario escribe la URL (ej. `https://example.com`) y hace clic en "Analizar".	4. El sistema valida el protocolo, resuelve el host y comprueba que la IP no sea local o privada (mitigando SSRF).
	5. El sistema realiza una petición HTTP GET externa y recupera las cabeceras del servidor.
	6. El sistema valida cabeceras críticas (CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Server), calcula la puntuación (restando 15 pts por cabecera ausente) e inicia el paso 6 del **Flujo Principal** .
Flujo Alternativo - FA3 (Analizar Repositorio de GitHub)	
Acción del Actor	Respuesta del Sistema
1. El usuario selecciona la opción "GitHub Repo" en el formulario.	2. El sistema habilita los campos de URL de repositorio y Token de acceso opcional.
3. El usuario escribe la URL de GitHub (y opcionalmente su token) y hace clic en "Analizar".	4. El sistema solicita a la API de GitHub la descarga de la rama principal del repositorio en formato comprimido ZIP. Si es privado, inyecta el token en la cabecera `Authorization`.
	5. El sistema procesa el ZIP directamente en memoria, descomprime sus archivos y aplica el análisis estático recursivo sobre archivos legibles de código (excluyendo carpetas pesadas).
	6. El sistema acumula las penalidades del análisis, calcula el puntaje final y ejecuta el paso 6 del **Flujo Principal** .
Flujo de Excepciones - FE1 (Archivo o campo de análisis vacío)	
Acción del Actor	Respuesta del Sistema
1. El usuario hace clic en "Analizar" sin seleccionar un archivo, sin pegar texto o con campos vacíos.	2. El sistema intercepta el formulario, detiene la ejecución del análisis y muestra una alerta: "Por favor selecciona un archivo" o "target_value es requerido".
Flujo de Excepciones - FE2 (Falla en lectura de archivo local)	
Acción del Actor	Respuesta del Sistema
1. El usuario sube un archivo binario o corrupto.	2. El sistema intenta leer el texto, captura el fallo de decodificación de caracteres y retorna un error HTTP 400 señalando formato inválido.
Flujo de Excepciones - FE3 (URL sin protocolo o inválida)	
Acción del Actor	Respuesta del Sistema
1. El usuario ingresa una dirección de sitio web sin prefijo (ej. `example.com`).	2. El sistema detecta la ausencia del protocolo `http://` o `https://`, bloquea la ejecución y muestra la advertencia: "URL inválida".
Flujo de Excepciones - FE4 (SSRF detectado en URL)	
Acción del Actor	Respuesta del Sistema
1. El usuario ingresa una dirección que apunta a la red local del servidor (ej. `http://localhost` o `127.0.0.1`).	2. El sistema resuelve la IP, identifica el rango local/privado, detiene la petición de red y devuelve la alerta: "Acceso denegado a rangos IP privados".
Flujo de Excepciones - FE5 (Host web caído o inactivo)	
Acción del Actor	Respuesta del Sistema



1. El usuario ingresa un dominio inactivo que no responde.	2. El sistema intenta conectarse y, tras agotar el tiempo de espera (timeout), cancela la operation y notifica la falla de red.
Flujo de Excepciones - FE6 (Repositorio privado de GitHub inaccesible)	
Acción del Actor	Respuesta del Sistema
1. El usuario ingresa la URL de un repositorio privado sin proporcionar un token correcto.	2. El sistema recibe una respuesta HTTP 404/401 de la API de GitHub, aborta la descarga y avisa: "No se pudo acceder al repositorio. Verifique que sea público o que el token sea correcto".
Flujo de Excepciones - FE7 (Cuota de API de GitHub agotada)	
Acción del Actor	Respuesta del Sistema
1. El usuario realiza múltiples descargas consecutivas anónimas de GitHub.	2. El sistema intercepta el código de error HTTP 403 (Rate Limit Exceeded) de GitHub y notifica al usuario la necesidad de iniciar sesión y configurar un Token personal.

Fuente: Elaboración propia

Tabla 2

Escenario: Identificar Dependencias Vulnerables

Escenario de caso de uso: Identificar dependencias vulnerables	
Tipo	Obligatorio (Incluido en CU01)
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Usuario
Descripción	Permite al sistema revisar las dependencias importadas en el código de desarrollo o listadas en un archivo de requerimientos (`requirements.txt`) para comprobar si tienen vulnerabilidades conocidas (CVEs).
Precondiciones	El usuario debe haber iniciado un escaneo de código (CU01) que contenga importaciones de librerías o un archivo `requirements.txt`.
Postcondiciones	Se genera un listado de dependencias vulnerables identificadas que se asocian como hallazgos de seguridad en el reporte generado.
Narrativa del Caso de Uso (Flujo Principal - Detección de CVEs)	
Acción del Actor	Respuesta del Sistema
1. El usuario inicia el escaneo del código fuente o repositorio.	2. El sistema detecta la presencia de importaciones de librerías comunes (como Flask, Django, requests) o la existencia de un archivo `requirements.txt`.
	3. El sistema analiza el nombre y la versión de cada dependencia listada.
	4. El sistema contrasta las versiones de las librerías contra su base de datos local de vulnerabilidades conocidas (CVEs).



	5. Si coincide con una versión vulnerable registrada, el sistema genera un hallazgo de severidad media/alta detallando la vulnerabilidad e indicando la remediación (actualizar librería) en el reporte final.
Flujo Alternativo - FA1 (Análisis automático en IDE al guardar archivo)	
Acción del Actor	Respuesta del Sistema
1. El desarrollador guarda un archivo de declaración de dependencias (`requirements.txt`) en el IDE de VS Code.	2. La extensión envía el archivo al backend de manera asíncrona. El sistema ejecuta la validación de CVEs en la base de datos de inmediato y retorna la lista de dependencias comprometidas para ser subrayadas en el IDE.
Flujo de Excepciones - FE1 (Archivo de dependencias vacío o ilegible)	
Acción del Actor	Respuesta del Sistema
1. El desarrollador adjunta o escanea un archivo `requirements.txt` corrupto o con formato no estándar.	2. El sistema intenta procesar el archivo, identifica que la estructura de líneas no cumple con el formato estándar `paquete==version`, omite el análisis de CVEs para las líneas mal estructuradas y continúa con el escaneo del resto del código.

Fuente: Elaboración propia



Tabla 3

Escenario: Generar y Validar Tokens de API

Escenario de caso de uso: Generar y validar tokens de API	
Tipo	Obligatorio
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Usuario
Descripción	Permite al usuario generar nuevos tokens de autenticación API para integraciones con su entorno local (IDE) o herramientas externas, y al sistema validar dichas credenciales en cada petición REST.
Precondiciones	El usuario debe estar logueado y encontrarse en el dashboard del sistema, o realizar una petición REST con una cabecera X-API-Key.
Postcondiciones	Se registra un nuevo token único en estado activo en la base de datos, o se valida exitosamente el token en una solicitud entrante.
Narrativa del Caso de Uso (Flujo Principal - Generar Token)	
Acción del Actor	Respuesta del Sistema
1. El usuario navega a la sección de "Tokens de API" en el dashboard.	2. El sistema muestra la lista de sus tokens y la opción de generar uno nuevo.
3. El usuario hace clic en el botón "Generar nuevo token".	4. El sistema genera una cadena aleatoria única y la asocia a la cuenta del usuario actual.
	5. El sistema registra el token, fecha de creación y estado activo en la base de datos (MySQL/SQLite).
	6. El sistema actualiza la lista y muestra en pantalla la clave generada para que el usuario pueda copiarla.
Flujo Alternativo - FA1 (Validar Token en peticiones de API)	
Acción del Actor	Respuesta del Sistema
1. Un cliente externo (o la extensión del IDE) envía una solicitud HTTP a la API REST del backend incluyendo la cabecera `X-API-Key` con el valor del token.	2. El sistema intercepta la cabecera de la petición, consulta la base de datos de tokens activos y valida que sea correcto.
	3. Si el token es válido y está activo, el sistema autoriza la operación y devuelve la respuesta correspondiente en formato JSON.
Flujo de Excepciones - FE1 (Token inválido o expirado)	
Acción del Actor	Respuesta del Sistema
1. Un cliente de integración externa envía una solicitud con una cabecera `X-API-Key` vacía, errónea o desactivada.	2. El sistema intercepta el token, realiza la búsqueda en base de datos y al no encontrar coincidencias válidas, rechaza la solicitud retornando un error HTTP 401 (No autorizado / Token inválido).

Fuente: Elaboración propia



Tabla 4

Escenario: Gestionar Sesiones Administrativas

Escenario de caso de uso: Gestionar sesiones administrativas	
Tipo	Obligatorio
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Administrador
Descripción	Permite controlar de forma segura el inicio de sesión, el cierre y la expiración automática del token de sesión web para el perfil de administración.
Precondiciones	El usuario debe tener una cuenta de administrador registrada en el sistema y encontrarse en la página de login.
Postcondiciones	Se establece una cookie segura `admin_session` en el navegador y se registra el ID de sesión activo en la base de datos.
Narrativa del Caso de Uso (Flujo Principal - Iniciar Sesión)	
Acción del Actor	Respuesta del Sistema
1. El administrador ingresa a la dirección `/login` en el navegador.	2. El sistema muestra la interfaz de inicio de sesión con campos para Nombre de Usuario y Contraseña.
3. El administrador rellena sus credenciales y presiona "Iniciar Sesión".	4. El sistema captura la contraseña, la hashea utilizando SHA-256 y verifica las credenciales contra la base de datos.
	5. Si son correctas, el sistema genera un ID de sesión criptográfico, lo guarda en la tabla `admin_sessions` especificando expiración en 6 horas y establece la cookie `admin_session` (con flags HttpOnly y SameSite=lax) redirigiendo al dashboard (`/dashboard`).
Flujo Alternativo - FA1 (Cerrar Sesión - Logout)	
Acción del Actor	Respuesta del Sistema
1. El administrador hace clic en el enlace "Cerrar sesión" en el menú de navegación superior.	2. El sistema captura el ID de sesión de la cookie, busca el registro en la base de datos y lo revoca/elimina.
	3. El sistema destruye la cookie de sesión en el cliente y redirige a la página principal del portal.
Flujo de Excepciones - FE1 (Credenciales de acceso incorrectas)	
Acción del Actor	Respuesta del Sistema
1. El administrador ingresa una contraseña o usuario incorrecto y presiona "Iniciar Sesión".	2. El sistema rechaza las credenciales hasheadas, detiene el flujo de creación de sesión y muestra un mensaje de error: "Usuario o contraseña incorrectos" con código de estado HTTP 401.

Fuente: Elaboración propia



Tabla 5

Escenario: Gestionar Cuentas de Usuario

Escenario de caso de uso: Gestionar cuentas de usuario	
Tipo	Obligatorio
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Administrador
Descripción	Permite al administrador crear nuevas cuentas de usuarios (con rol de usuario común o administrador) y eliminar cuentas del sistema.
Precondiciones	El administrador debe haber iniciado sesión en el sistema y encontrarse en el panel de control de usuarios.
Postcondiciones	Las cuentas de usuario son creadas o eliminadas en la base de datos sqlite del sistema de forma persistente.
Narrativa del Caso de Uso (Flujo Principal - Crear/Eliminar por el Admin)	
Acción del Actor	Respuesta del Sistema
1. El administrador accede a la pestaña de "Usuarios" en el Dashboard.	2. El sistema muestra la lista de usuarios activos y el formulario de registro de nuevas cuentas.
3. El administrador rellena los campos (Nombre de Usuario, Email y Contraseña) y presiona "Crear".	4. El sistema valida que los campos requeridos no estén vacíos y que el nombre de usuario sea único.
	5. El sistema hasha la contraseña proporcionada utilizando el algoritmo criptográfico SHA-256 y guarda la cuenta con rol `admin` en la tabla `users`.
6. El administrador pulsa el botón "Eliminar" asociado al usuario que desea borrar de la lista.	7. El sistema valida que el nombre no corresponda al administrador maestro (`admin`) ni al usuario logueado en la sesión activa. Si pasa, ejecuta el borrado SQL y recarga la interfaz.
Flujo Alternativo - FA1 (Crear usuario general desde el formulario de registro público)	
Acción del Actor	Respuesta del Sistema
1. Un usuario visitante ingresa a la ruta `/register` de la web en lugar de hacerlo desde el dashboard administrativo.	2. Muestra la pantalla pública de registro de usuarios generales.
3. El visitante rellena el formulario de registro con sus datos y presiona "Registrarse".	4. El sistema valida los campos, hasha la contraseña, guarda la cuenta con rol general de tipo `user` por defecto en la base de datos, y redirige a la pantalla de login.
Flujo de Excepciones - FE1 (Intento de registrar un nombre de usuario duplicado)	
Acción del Actor	Respuesta del Sistema
1. El administrador (o visitante) intenta crear un usuario cuyo nombre ya está registrado en la base de datos.	2. El sistema detecta la colisión del nombre en la base de datos, cancela la inserción SQL, detiene el proceso y muestra una alerta: "El usuario ya está registrado".

Fuente: Elaboración propia



Tabla 6

Escenario: Exportar Reportes de Seguridad

Escenario de caso de uso: Exportar reportes de seguridad	
Tipo	Obligatorio
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Usuario
Descripción	Permite al usuario descargar los resultados del análisis de seguridad en un archivo PDF estructurado (para documentación) o un archivo JSON (para integraciones).
Precondiciones	El reporte solicitado debe estar registrado en el historial de base de datos del sistema.
Postcondiciones	Se envía un flujo de descarga del archivo solicitado (`reporte-seguridad-{id}.pdf` o `.json`) al navegador del usuario.
Narrativa del Caso de Uso (Flujo Principal - Exportar a PDF)	
Acción del Actor	Respuesta del Sistema
1. El usuario hace clic en el botón "Exportar PDF" en la cabecera de la vista de un reporte detallado.	2. El sistema lee el ID del escaneo solicitado desde la ruta del navegador.
	3. El sistema realiza la consulta a la base de datos para obtener los metadatos y hallazgos vinculados al escaneo.
	4. El sistema invoca al servicio de generación de PDF para crear el documento con tablas y esquemas de color de severidad.
	5. El sistema añade la cabecera `Content-Disposition: attachment` y retorna el archivo como un flujo descargable (`StreamingResponse`).
Flujo Alternativo - FA1 (Descargar el reporte en formato JSON)	
Acción del Actor	Respuesta del Sistema
1. El usuario hace clic en el botón "Exportar JSON" en la vista del reporte.	2. El sistema intercepta la petición, lee el ID y recupera los datos del escaneo desde la base de datos.
	3. Mapea y serializa los datos del escaneo (`id`, `target_type`, `score`, `findings`) a una cadena JSON estructurada.
	4. El sistema añade la cabecera `Content-Disposition: attachment` y retorna el archivo al navegador en formato JSON legible (`JSONResponse`).
Flujo de Excepciones - FE1 (Reporte no registrado en base de datos)	
Acción del Actor	Respuesta del Sistema
1. El usuario modifica la URL solicitando exportar un ID de escaneo	2. El sistema realiza la búsqueda en base de datos, no encuentra ningún registro, aborta el flujo de generación del



inexistente (ej. `reports/99999/export-pdf`).	archivo y retorna un error HTTP 404 (Scan no encontrado).
---	---

Fuente: Elaboración propia

Tabla 7

Escenario: Visualizar Diagnósticos en el IDE

Escenario de caso de uso: Visualizar diagnósticos en el IDE	
Tipo	Obligatorio
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Usuario
Descripción	Permite al desarrollador observar los fallos de seguridad detectados de manera directa sobre las líneas de su código en tiempo real en VS Code.
Precondiciones	La extensión debe estar instalada en VS Code, configurada con un token de API válido y el servidor backend debe estar en ejecución.
Postcondiciones	El editor de VS Code dibuja subrayados coloreados (rojo para severidades altas, amarillo para medias) sobre las líneas vulnerables detectadas.
Narrativa del Caso de Uso (Flujo Principal - Subrayados en el Editor)	
Acción del Actor	Respuesta del Sistema
1. El desarrollador modifica un archivo de código local y lo guarda.	2. La extensión de VS Code detecta el evento de guardado en segundo plano de manera asíncrona.
	3. La extensión envía el contenido del código y el token al servidor backend.
	4. El servidor procesa las reglas OWASP, identifica fallas y responde con los hallazgos en JSON.
	5. La extensión recibe el JSON, borra marcas antiguas y dibuja subrayados de color (rojo/amarillo) en las líneas afectadas del editor de código.
Flujo Alternativo - FA1 (Visualizar hallazgos en la barra lateral - Dashboard)	
Acción del Actor	Respuesta del Sistema
1. El desarrollador abre la barra lateral de la extensión en lugar de mirar los subrayados.	2. Abre un panel Webview interactivo que agrupa y lista todas las vulnerabilidades detectadas del proyecto.
3. El desarrollador hace clic sobre un archivo listado con vulnerabilidades en el panel.	4. El sistema abre el archivo enfocando el cursor en la línea del hallazgo y muestra la sugerencia de remediación.
Flujo de Excepciones - FE1 (Diagnósticos desactivados por configuración)	
Acción del Actor	Respuesta del Sistema



1. El desarrollador tiene definida la configuración <code>`owaspVerificator.showDiagnostics`</code> en falso (<code>`false`</code>).	2. El sistema solicita y procesa el análisis con el backend de forma oculta, pero restringe la renderización de subrayados de error en las pestañas de edición del código.
--	--

Fuente: Elaboración propia

Tabla 8

Escenario: Guiar Remediación por Inteligencia Artificial

Escenario de caso de uso: Guiar remediación por Inteligencia Artificial	
Tipo	Opcional (Extendida de CU01)
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Usuario
Descripción	Permite al desarrollador ver soluciones de remediación contextualizadas e integrarlas de forma directa en herramientas de IA (Copilot/Gemini).
Precondiciones	Se debe haber ejecutado un análisis y tener abiertos los resultados (reporte web o panel del IDE).
Postcondiciones	El sistema muestra los pasos de remediación ajustados y entrega un prompt de ingeniería listo para copiar en el portapapeles.
Narrativa del Caso de Uso (Flujo Principal - Remediación desde la Web)	
Acción del Actor	Respuesta del Sistema
1. El desarrollador hace clic sobre un hallazgo listado en el reporte web detallado.	2. Muestra en pantalla el bloque de código que causó la penalidad.
3. El desarrollador presiona el botón "Ver Remediación" o "Generar Prompt de IA".	4. Ejecuta el detector de importaciones del archivo analizado buscando frameworks conocidos (FastAPI, Flask, Express, Django).
	5. Recupera la plantilla de remediación OWASP correspondiente para ese framework y compone un prompt estructurado de código.
	6. Renderiza los pasos de solución en pantalla y copia automáticamente el prompt generado en el portapapeles.
Flujo Alternativo - FA1 (Solicitar remediación desde la bombilla Quick Fix de VS Code)	
Acción del Actor	Respuesta del Sistema
1. El desarrollador coloca el cursor sobre la línea subrayada en VS Code y abre la bombilla (Quick Fix / <code>`Ctrl+.`</code>).	2. El sistema despliega el menú de acciones rápidas de la extensión.
3. El desarrollador selecciona la opción "OWASP: Generar Prompt de Solución".	4. El sistema ejecuta la consulta de remediación adaptada de forma local y coloca el prompt formateado en el portapapeles del editor para usar con Copilot o Gemini.
Flujo de Excepciones - FE1 (Imposibilidad de identificar el framework - Remediación Genérica)	



Acción del Actor	Respuesta del Sistema
1. El desarrollador solicita la remediación sobre un código sin importaciones explícitas de frameworks soportados.	2. Detecta que no puede asociar un framework web, utiliza la plantilla de remediación estándar del lenguaje y compone el prompt usando la sintaxis genérica por defecto.

Fuente: Elaboración propia

Tabla 9

Escenario: Monitorear y Auditar Accesos

Escenario de caso de uso: Monitorear y auditar accesos	
Tipo	Obligatorio
Versión	V 1.0
Autor(es)	Concha Llaca, Gerardo Alejandro
Actores	Administrador
Descripción	Guarda un registro cronológico de peticiones HTTP en el servidor backend para auditoría e inspección de red, mostrando vistas detalladas en el panel administrativo.
Precondiciones	El administrador debe haber iniciado sesión en el sistema y encontrarse en el dashboard web.
Postcondiciones	Se listan los registros de auditoría de red depurados o secuenciales en pantalla.
Narrativa del Caso de Uso (Flujo Principal - Monitoreo por IP única)	
Acción del Actor	Respuesta del Sistema
1. El administrador hace clic en la pestaña "Monitoreo" del panel lateral administrativo.	2. El sistema consulta las marcas temporales e IPs almacenadas en la tabla `access_logs` de la base de datos (MySQL/SQLite).
	3. El sistema recorre los registros en orden descendente y aplica un filtro para dedupilar las direcciones IP repetidas, conservando solo la visita más reciente de cada IP única.
	4. El sistema renderiza la tabla de monitoreo mostrando: Dirección IP, Ruta del Recurso, User-Agent, Fecha y Hora del último acceso.
Flujo Alternativo - FA1 (Visualizar listado secuencial completo)	
Acción del Actor	Respuesta del Sistema
1. El administrador navega a la sección inferior de "Accesos Recientes" en la pantalla principal del Dashboard.	2. El sistema consulta las últimas 100 peticiones registradas de forma cronológica sin aplicar filtros de duplicidad.
	3. El sistema carga y muestra secuencialmente la lista completa en una tabla compacta para auditoría rápida de ráfagas de red.
Flujo de Excepciones - FE1 (Pérdida de privilegios o sesión)	
Acción del Actor	Respuesta del Sistema



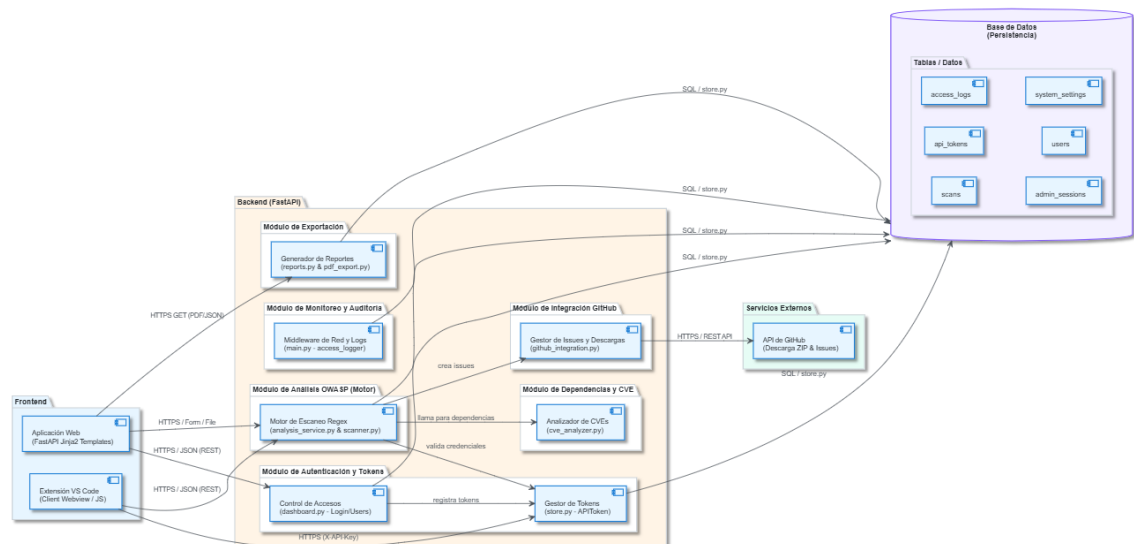
- | | |
|---|--|
| 1. El administrador intenta ingresar a `/monitoring` con la sesión web expirada o alterando su rol. | 2. El sistema intercepta la petición en los controladores de ruta de FastAPI, valida que no posee permisos válidos de sesión, bloquea el acceso a la base de datos y redirige a la pantalla de login `/login`. |
|---|--|

Fuente: Elaboración propia

2.2 Vista Lógica

Figura 1

Vista Lógica del Sistema



Fuente: Elaboración propia

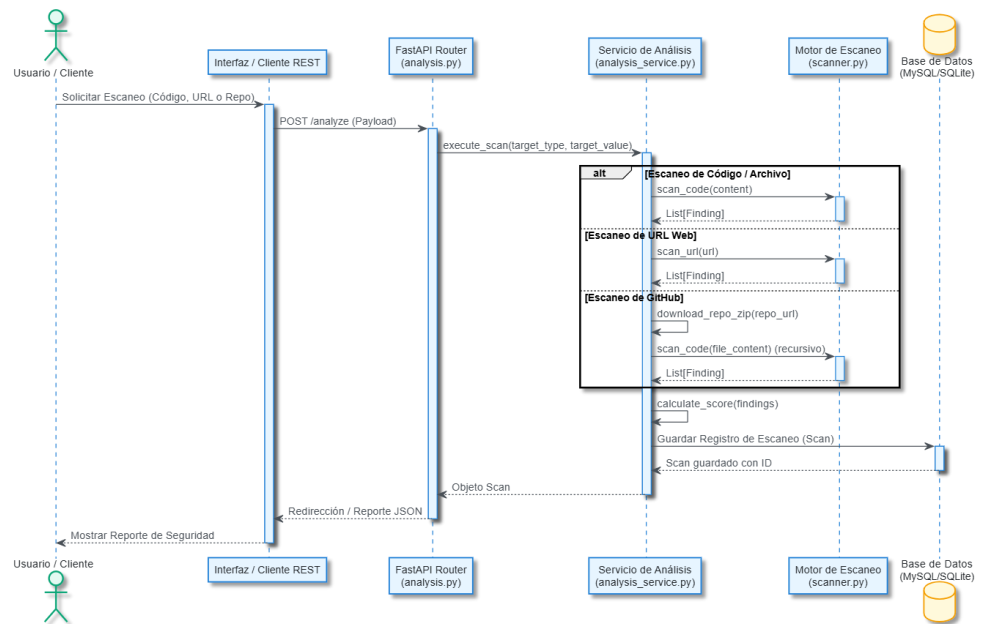


2.3 Vista del Proceso

Diagrama de Secuencia

- **Diagrama de secuencia- Escanear objetivos**

Figura 16: Diagrama de secuencia - Escanear objetivos

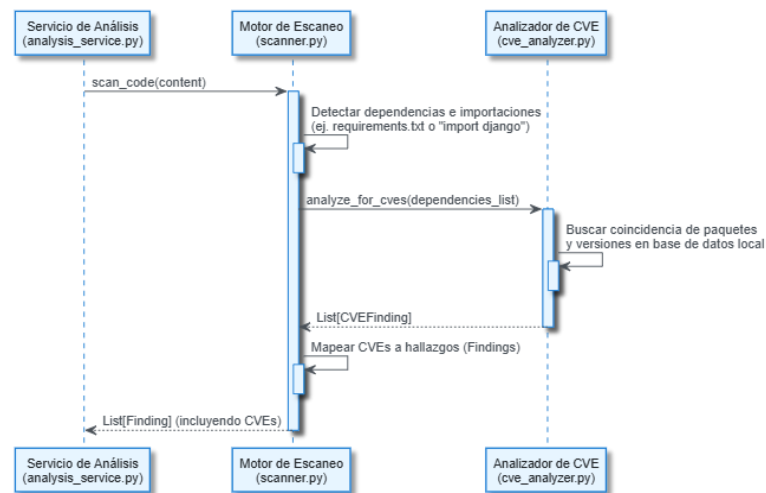


Fuente: Elaboración Propia

- **Diagrama de secuencia- Identificar dependencias**

Figura 17: Diagrama de secuencia - Identificar

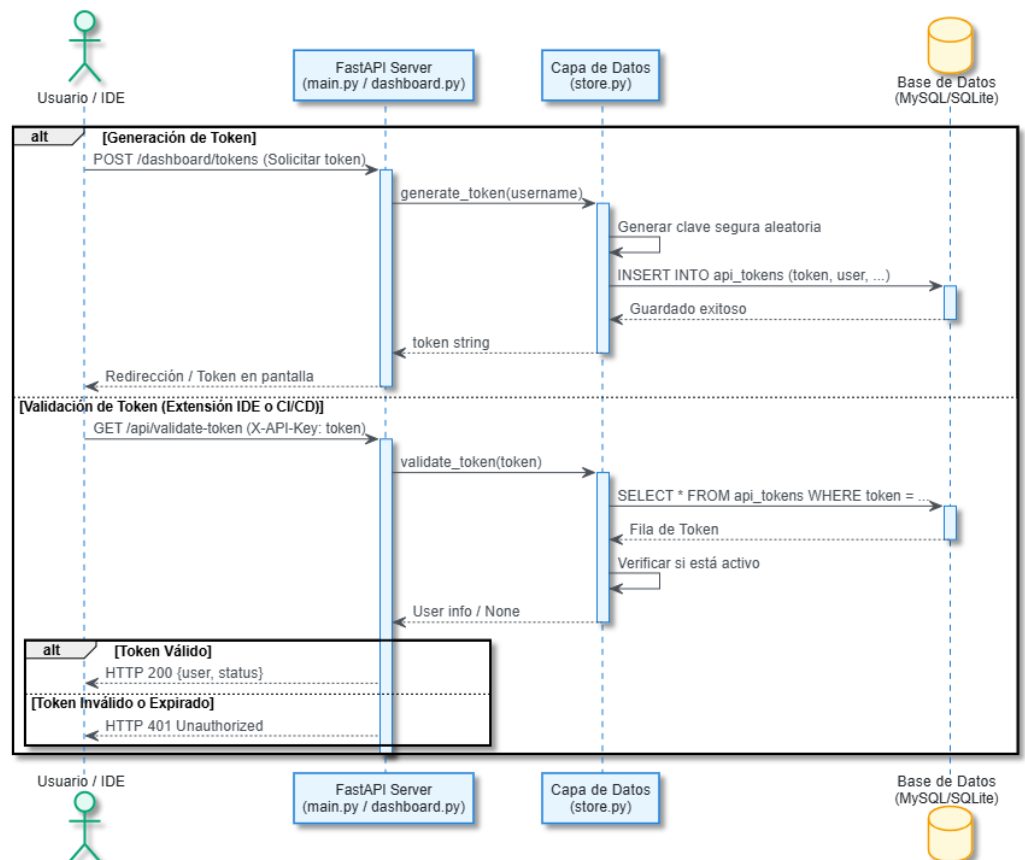
dependencias



Fuente: Elaboración Propia

- **Diagrama de secuencia- Generar y validar tokens de API**
Figura 18: Diagrama de secuencia - Generar y validar tokens de

API

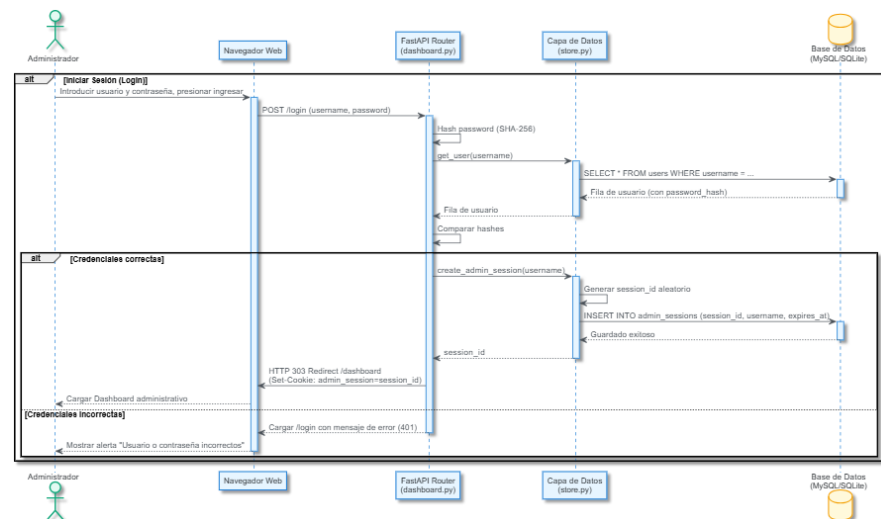


Fuente: Elaboración propia



- **Diagrama de secuencia- Gestionar sesiones administrativas**

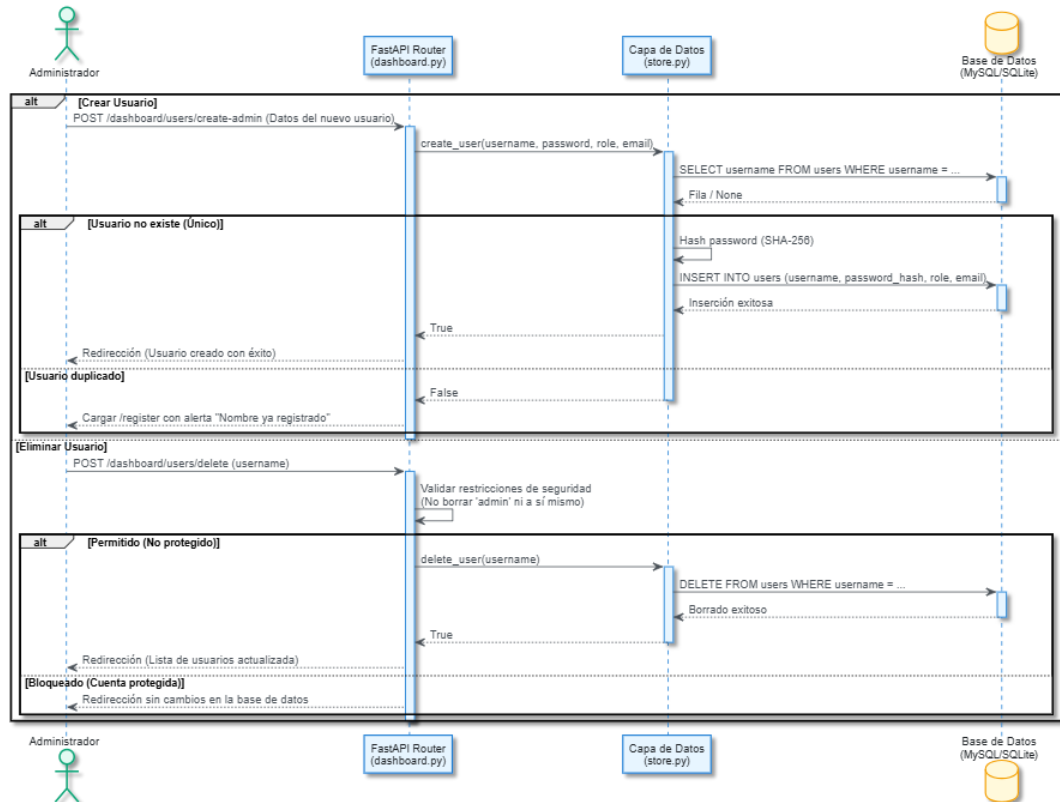
Figura 19: Diagrama de secuencia - Gestionar sesiones administrativas



Fuente: Elaboración propia

- **Diagrama de secuencia- Gestionar usuarios**

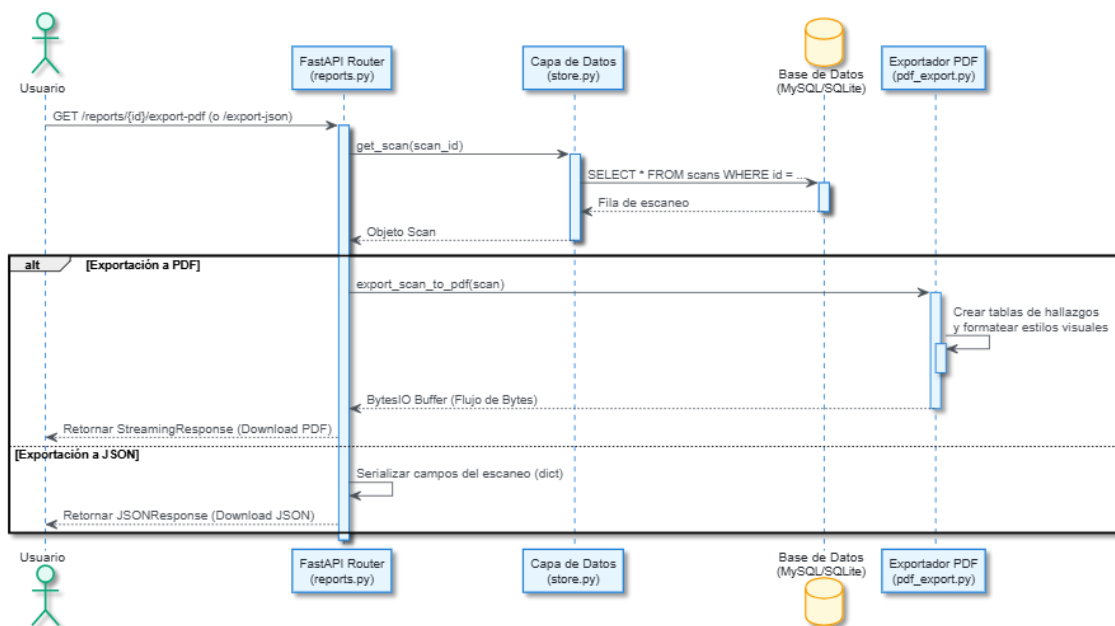
Figura 20: Diagrama de secuencia - Gestionar usuarios



Fuente: Elaboración propia

○ Diagrama de secuencia- Exportar reportes detallados

Figura 21: Diagrama de secuencia - Exportar reportes detallados

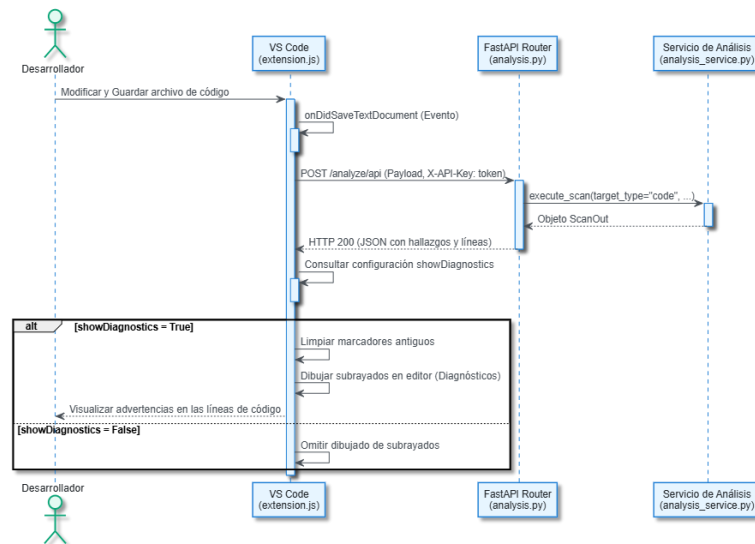




Fuente: Elaboración propia

- **Diagrama de secuencia- Visualizar diagnósticos en IDE**
Figura 22: Diagrama de secuencia - Visualizar diagnósticos en

IDE

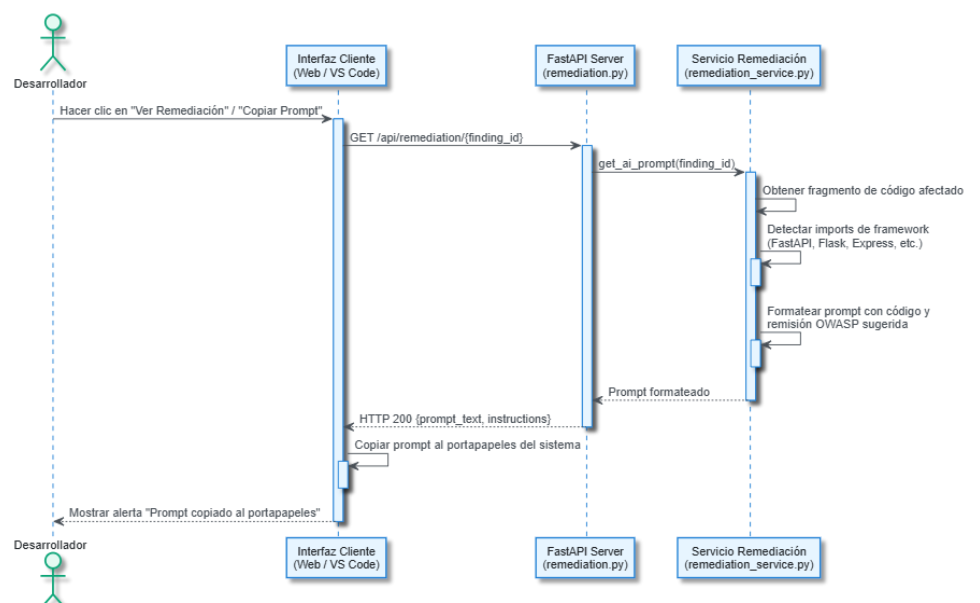


Fuente: Elaboración propia

- **Diagrama de secuencia- Guiar remdiación por Inteligencia Artificial**
Artificial

Figura 23: Diagrama de secuencia - Guiar remediación por

Inteligencia Artificial

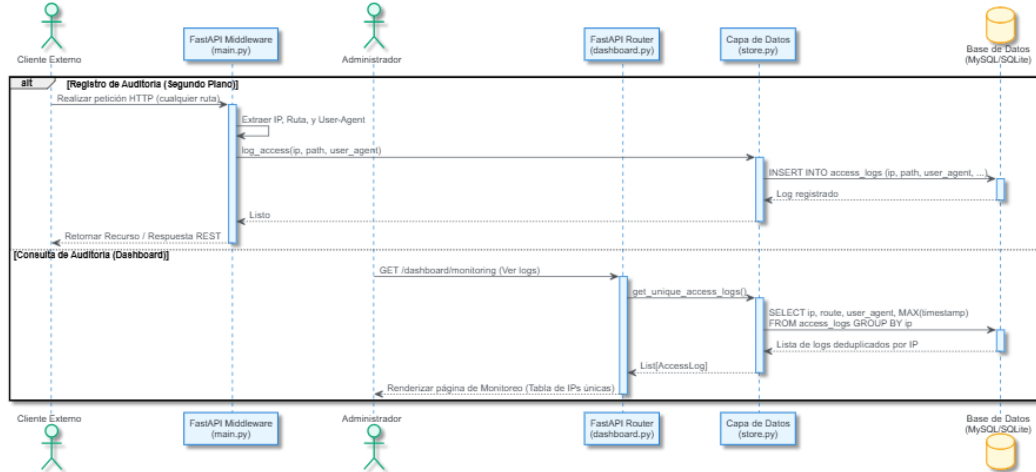




Fuente: Elaboración propia

- **Diagrama de secuencia- Monitorear y auditar accesos**

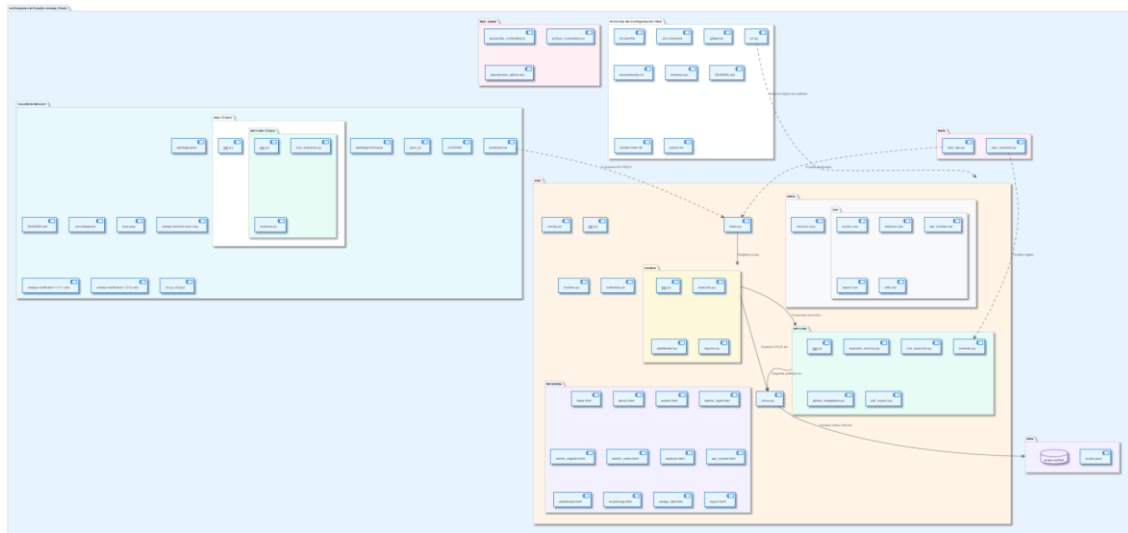
Figura 24: Diagrama de secuencia- Monitorear y auditar accesos



Fuente: Elaboración propia

2.4 Vista del Desarrollo

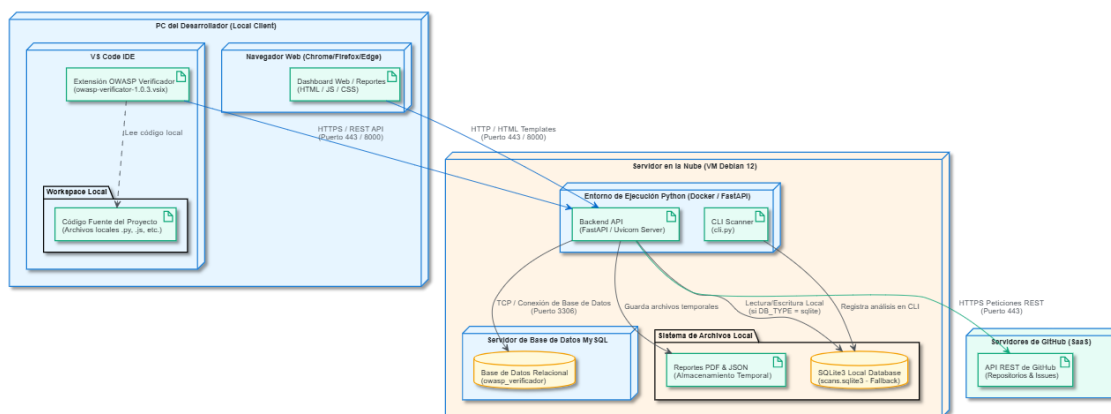
Figura N°36: Diagrama de Paquetes



Fuente: Elaboración propia

2.5 Vista Física

Figura N°37: Diagrama de Despliegue



Fuente: Elaboración propia

3. Objetivos y Limitaciones Arquitectónicas

3.1 Rendimiento y Tiempos de Respuesta (Basado en RNF01)

- **Objetivo:** Garantizar que el motor de análisis estático responda de forma ágil para no interrumpir el flujo continuo de codificación del desarrollador.
- **Descripción (tácticas):**
 - Compilación previa en memoria de las expresiones regulares (Regex) de las reglas OWASP durante el inicio del servidor FastAPI.
 - Uso de funciones asíncronas (**async/await**) en controladores para evitar el bloqueo del bucle de eventos ante solicitudes de red.
 - Delegación de tareas pesadas (como la descarga y descompresión de repositorios de GitHub) a servicios específicos no interactivos.
- **Limitaciones:**
 - El análisis de repositorios extensos de GitHub o el escaneo de URLs remotas con alta latencia de red puede exceder el límite establecido (15s); requiere implementar timeouts controlados de respuesta HTTP.

3.2 Seguridad de Acceso (Basado en RNF02)

- **Objetivo:** Proteger los endpoints de auditoría y la base de datos relacional ante accesos maliciosos o no autorizados.
- **Descripción (tácticas):**
 - Validación obligatoria de tokens de API en la cabecera **X-API-Key** mediante un middleware interceptor en FastAPI para endpoints sensibles.



- Hasheo criptográfico robusto (SHA-256) de contraseñas administrativas antes de su almacenamiento en la base de datos MySQL/SQLite3.
- Expiración y remoción automática en memoria de los tokens de sesión web administrativa pasadas 6 horas.
- **Limitaciones:**
 - La consulta de validación de tokens en cada petición REST puede introducir latencias menores de red; requiere la optimización e indexación de las consultas en la base de datos de tokens.

3.3 Arquitectura Desacoplada (Basado en RNF03)

- **Objetivo:** Garantizar la modularidad y separación estricta de responsabilidades para facilitar el mantenimiento y evitar el acoplamiento tecnológico.
- **Descripción (tácticas):**
 - Aislamiento completo de la lógica de persistencia de datos mediante el componente centralizado `InMemoryScanStore` en `store.py`.
 - Separación de código en capas independientes: Presentación (HTML/Vistas de IDE), Controladores (FastAPI Routers), Capa de Negocio (Módulos de Servicio) y Acceso a Datos.
- **Limitaciones:**
 - El desacoplamiento obliga al uso constante de objetos e interfaces de transferencia intermedios (esquemas Pydantic), lo que incrementa la cantidad de archivos a mantener en el repositorio.

3.4 Portabilidad y Compatibilidad (Basado en RNF04)

- **Objetivo:** Asegurar que el sistema se ejecute de forma idéntica en entornos locales de desarrollo (Windows/macOS) y servidores de producción en la nube (Debian 12).
- **Descripción (tácticas):**
 - Implementación de soporte relacional dual en `store.py` que detecta y conmuta automáticamente entre SQLite3 (archivo local plano) y MySQL (remoto en producción).
 - Desarrollo exclusivo bajo lenguajes multiplataforma (Python 3 y JavaScript para VS Code), evitando dependencias binarias propietarias de un sistema operativo específico.
- **Limitaciones:**
 - Para garantizar la persistencia dual transparente, se restringen las consultas de base de datos a sintaxis estándar SQL ANSI, impidiendo el uso de funciones específicas propietarias de MySQL.

3.5 Usabilidad e Interfaz Adaptativa (Basado en RNF05)

- **Objetivo:** Proveer una experiencia de desarrollo premium, visualmente atractiva y fluida que se adapte al tema del editor del programador.
- **Descripción (tácticas):**
 - Empleo de estilos premium de *glassmorphism* basados en variables CSS con tonalidades HSL coherentes con el tema activo de VS Code.



- Uso de la API nativa de diagnósticos de Microsoft para subrayar directamente las líneas con errores en la pestaña de problemas del editor.
- **Limitaciones:**
 - La extensión depende de los recursos de hardware y la renderización del panel de Webview integrado de VS Code (Chromium), lo que en computadoras de bajos recursos puede generar micro-retrasos en la carga inicial.

3.6 Extensibilidad del Motor (Basado en RNF06)

- **Objetivo:** Facilitar la incorporación de nuevas firmas y reglas de seguridad de forma modular sin alterar el flujo de los analizadores.
- **Descripción (tácticas):**
 - Estructuración del catálogo de reglas en una lista ordenada de diccionarios (**RULES**) modular dentro del archivo core **scanner.py**.
 - Carga dinámica e independiente de expresiones regulares para evaluar código fuente multiframework.
- **Limitaciones:**
 - Expresiones regulares mal estructuradas o mal probadas pueden causar un "backtracking catastrófico" y colgar el hilo del servidor web; requiere pruebas unitarias estrictas en **test_scanner.py** para cada regla nueva.

4. Análisis de Requerimientos

4.1 Requerimientos Funcionales

Tabla N°15: Cuadro de requerimientos funcionales final

ID	Nombre del Requerimiento	Descripción	Prioridad
RF01	Escanear objetivos	El sistema recibe código, archivo local, URL web o URL de repositorio de GitHub para su análisis.	Alta
RF02	Identificar dependencias	El sistema analiza dependencias declaradas y las coteja contra una lista predefinida de CVEs.	Alta
RF03	Generar y validar tokens de API	El sistema expone endpoints seguros y gestiona tokens de acceso para clientes externos.	Alta
RF04	Gestionar sesiones administrativas	El sistema administra las sesiones y tokens temporales de login/logout del administrador.	Alta
RF05	Gestionar usuarios	El administrador crea, edita, elimina usuarios y asigna roles.	Alta
RF06	Exportar reportes detallados	El sistema renderiza y exporta reportes interactivos en formato PDF y JSON.	Alta
RF07	Visualizar diagnósticos en IDE	La extensión de VS Code subraya líneas vulnerables directamente en el editor al guardar.	Alta



RF08	Guiar remediación en IDE	La extensión muestra remediaciones y sugerencias de código recomendadas según el framework.	Media
RF09	Monitorear y auditar accesos	El sistema registra el log de accesos de red (IP, ruta, usuario y fecha) de forma persistente.	Media

Fuente: Elaboración propia

4.2 Requerimientos No Funcionales

tabla n 16: Requerimientos No Funcionales

ID	Requerimiento No Funcional	Descripción / Criterio de Medida
RNF01	Rendimiento	El tiempo de escaneo de código local debe ser menor a 5s, y de URLs menor a 15s.
RNF02	Seguridad de Acceso	Los endpoints de auditoría y generación de tokens requieren cabecera X-API-Key.
RNF03	Arquitectura Desacoplada	Separación estricta entre presentación, negocio (servicios) y acceso a datos.
RNF04	Portabilidad	Debe poder ejecutarse en Windows, Linux y macOS sin requerir complejas bases de datos pesadas (SQLite/MySQL dual).
RNF05	Usabilidad e Interfaz	El panel del IDE debe emplear estilos premium glassmorphism y ser totalmente adaptativo.
RNF06	Extensibilidad	El motor estático de escaneo Regex debe basarse en un archivo

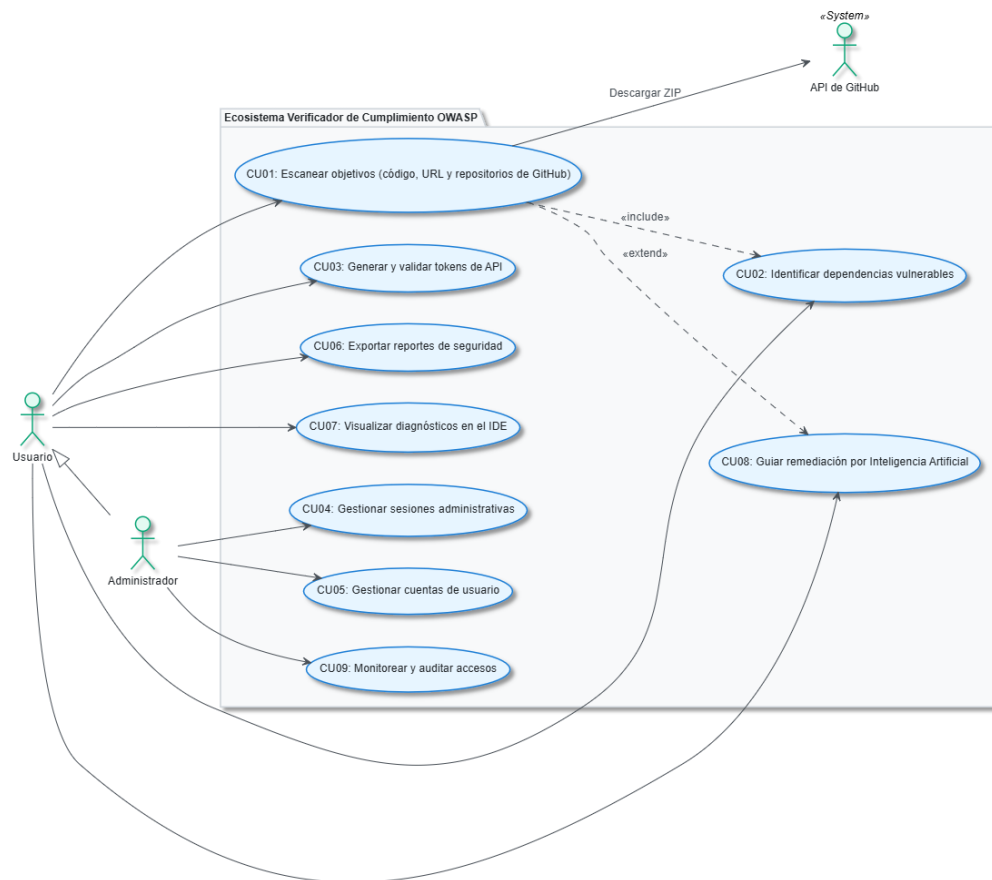


		modular de firmas extensible.
--	--	-------------------------------

Fuente: Elaboración propia

5. Vistas de Caso de Uso

Figura N°38: Diagrama de caso de uso general

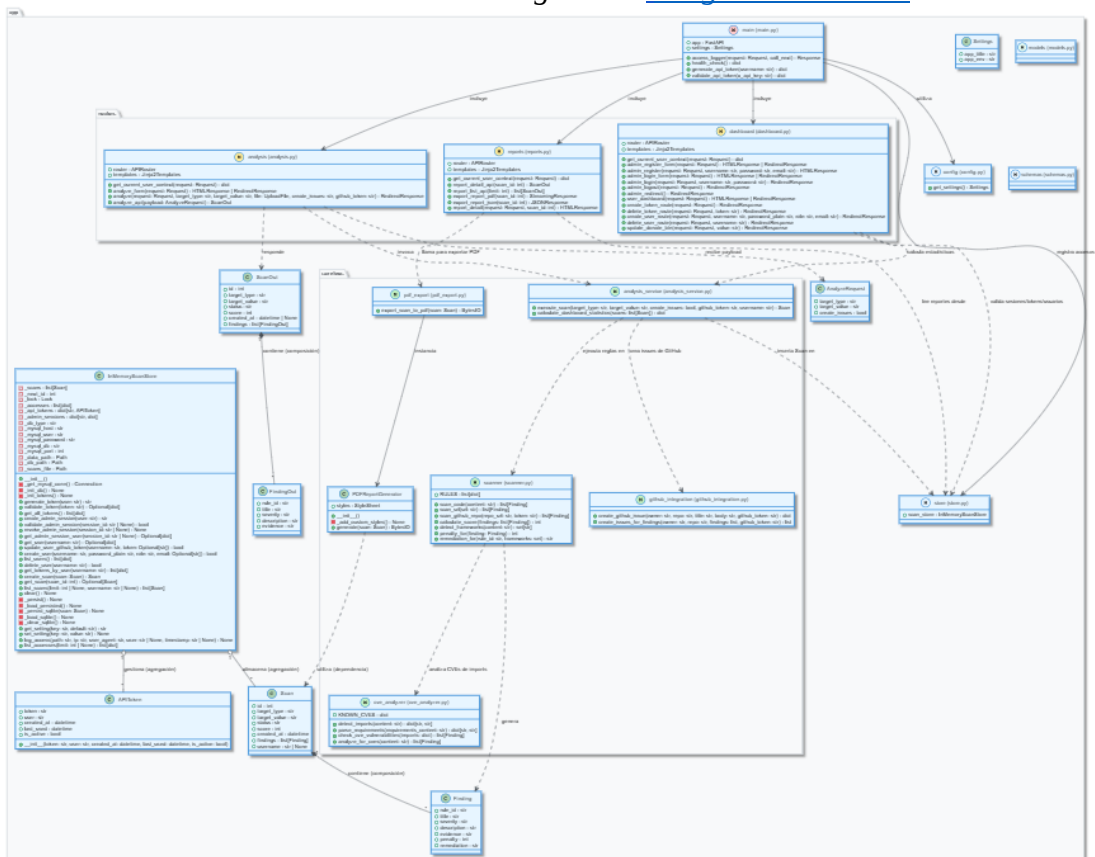


Fuente: Elaboración propia

6. Vista Lógica

6.1 Diagrama Contextual

Figura 25: Diagrama de clases

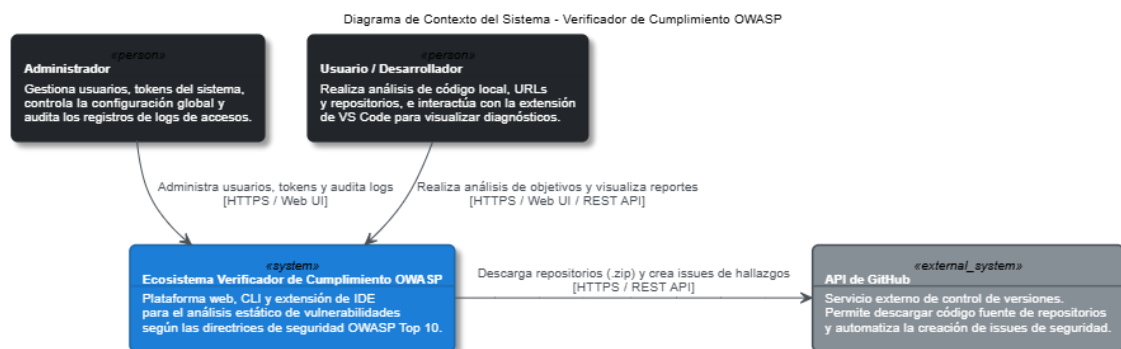


Fuente: Elaboración propia



6.2. Diagrama Contextual

Figura N°40: Diagrama Contextual

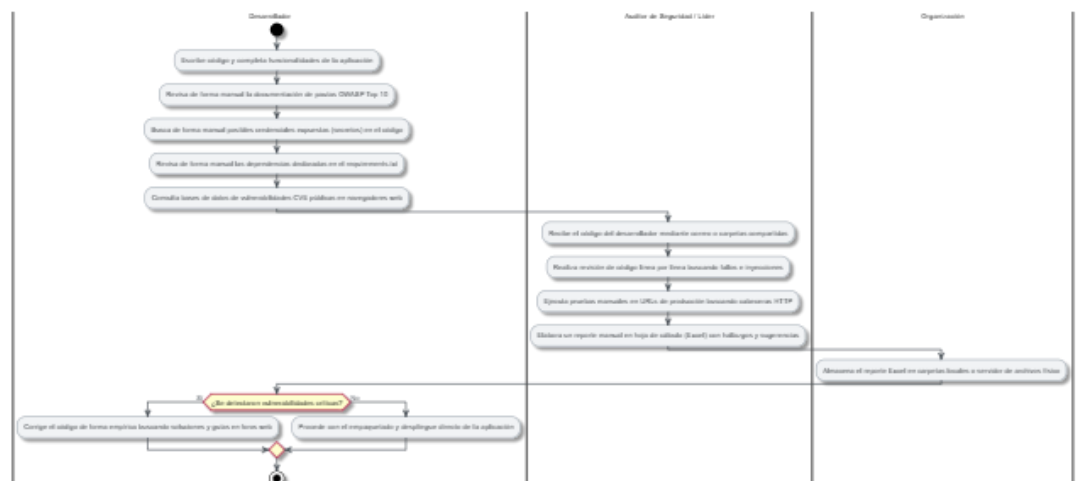


Fuente: Elaboración propia

7. Vista de Procesos

7.1 Diagrama de Proceso Actual

Figura N°41: Diagrama de Actividades Proceso Actual

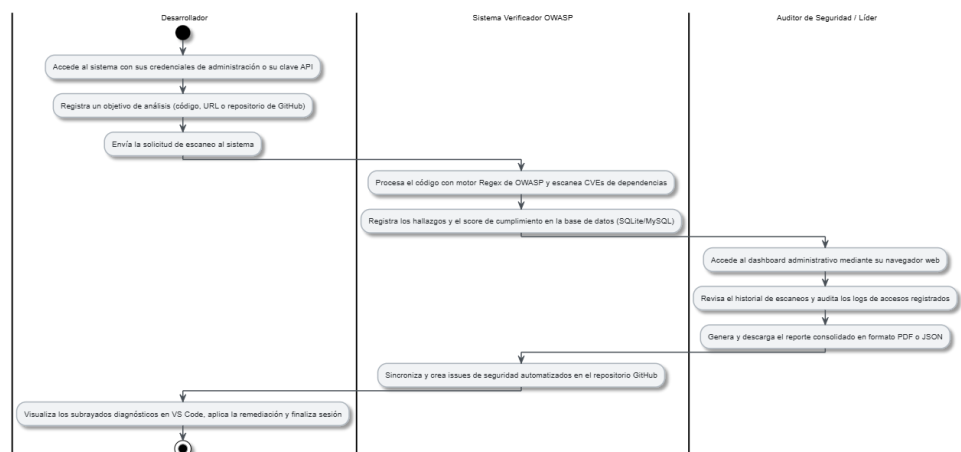




Fuente: Elaboración propia

7.2 Diagrama de Proceso Propuesto

Figura N°42: Diagrama de Actividades Proceso propuesto

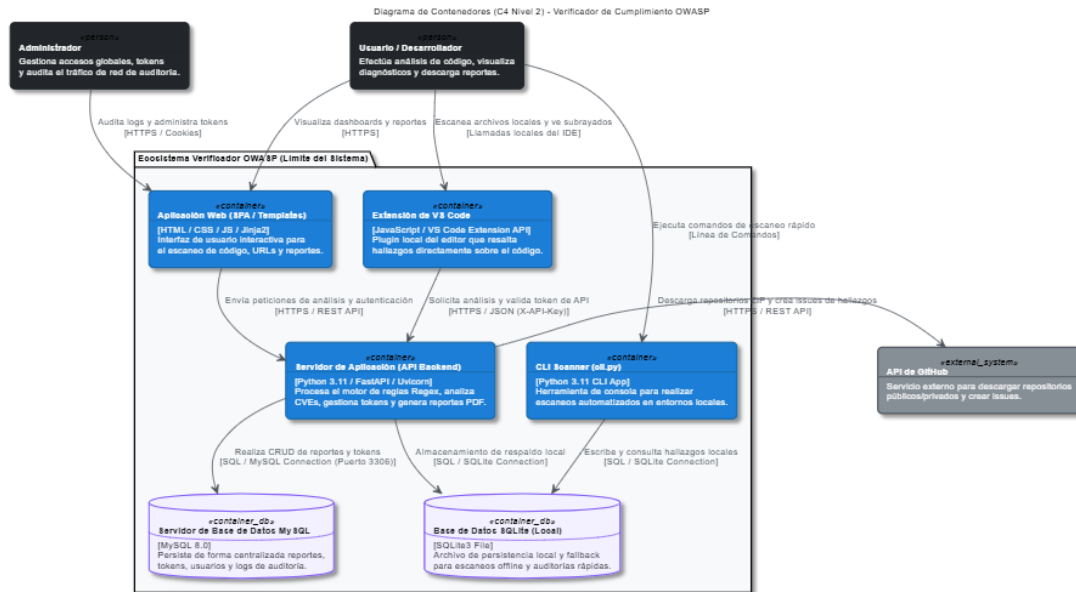


Fuente: Elaboración propia

8. Vista de Despliegue

8.1 Diagrama de Contenedor

Figura N°43: Diagrama de contenedor

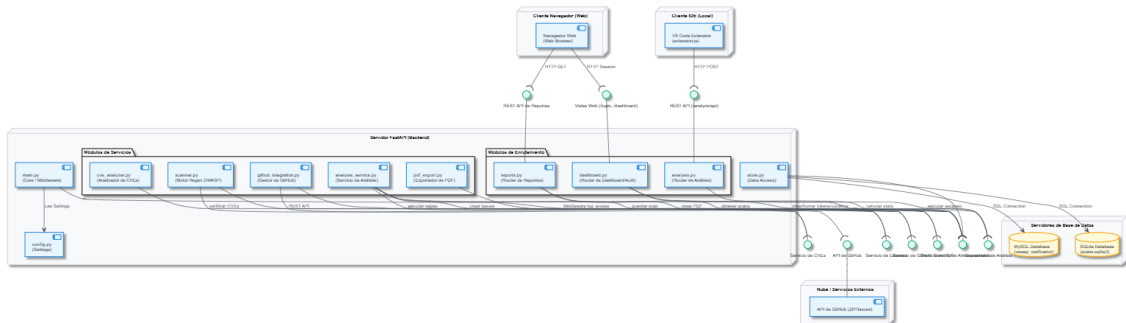


Fuente: Elaboración propia

9. Vista de Implementación

9.1 Diagrama de Componentes

Figura N°44: [Diagrama de componentes](#)

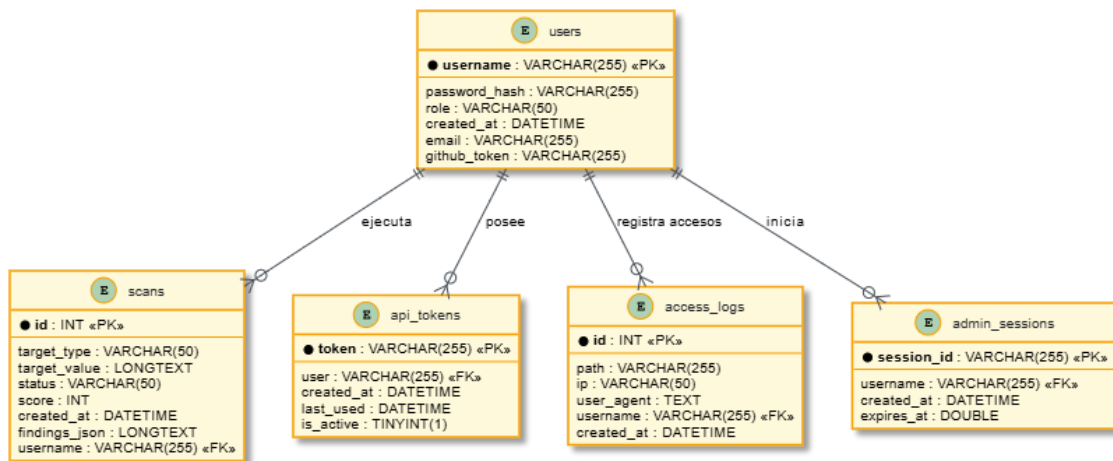


Fuente: Elaboración propia

10. Vista de Datos

10.1 Diagrama de Base de Datos

Figura N°45: [Diagrama de Base de Datos](#)



Fuente: Elaboración propia

11. Calidad

11.1 Escenario de Rendimiento (Basado en RNF01)

- **Estímulo:** Solicitud de escaneo de un archivo de código local o una dirección URL externa en producción.
- **Fuente:** Desarrollador interactivo.
- **Artefacto:** Extensión de VS Code (cliente) y API REST de análisis (`/api/v1/analyze` en el backend).
- **Entorno:** Edición de código en caliente en el IDE o auditoría en caliente desde el navegador.
- **Respuesta:**
 - Compilación y precarga de firmas Regex de OWASP en memoria al arrancar la aplicación.
 - Ejecución asíncrona de las solicitudes HTTP para evaluar cabeceras.
- **Métrica:**
 - Tiempo de análisis y respuesta para código local < 5 segundos.
 - Tiempo de análisis y respuesta para URLs externas < 15 segundos.



11.2 Escenario de Seguridad de Acceso (Basado en RNF02)

- **Estímulo:** Intento de consumo de endpoints sensibles del backend sin credenciales válidas.
- **Fuente:** Usuario malintencionado o script de escaneo automatizado externo.
- **Artefacto:** Endpoint de auditoría y generación de tokens de API.
- **Entorno:** Sistema en producción con autenticación activa.
- **Respuesta:**
 - Intercepción y rechazo inmediato de la petición (retorno de código HTTP 401) si no se provee la cabecera **X-API-Key** autorizada.
 - Registro de auditoría con la IP, ruta solicitada y timestamp en la base de datos de logs de accesos (**LogAccess**).
- **Métrica:**
 - 0 accesos no autorizados exitosos a la base de datos y endpoints administrativos.

11.3 Escenario de Mantenibilidad / Arquitectura Desacoplada (Basado en RNF03)

- **Estímulo:** Requerimiento de cambiar la persistencia de datos (ej: de SQLite3 local a MySQL en la nube) sin afectar el motor de análisis.
- **Fuente:** Desarrollador backend o arquitecto de software.
- **Artefacto:** Capa de acceso a datos (**store.py**).
- **Entorno:** Despliegue y mantenimiento del software.
- **Respuesta:**
 - Modificación exclusiva de la clase **InMemoryScanStore** en **store.py**.
 - La lógica de negocio (**scanner.py**, **cve_analyzer.py**) y de ruteadores (**routers/**) permanece 100% intacta.
- **Métrica:**
 - Tiempo de refactorización < 1 hora.
 - 0 líneas de código modificadas fuera del archivo **store.py**.

11.4 Escenario de Portabilidad (Basado en RNF04)

- **Estímulo:** Instalación y despliegue del sistema en un sistema operativo diferente (ej: migración de Windows local a máquina virtual Debian en la nube).
- **Fuente:** Administrador de sistemas o ingeniero de despliegue (DevOps).
- **Artefacto:** Servidor de aplicación y base de datos relacional.
- **Entorno:** Despliegue multiplataforma.
- **Respuesta:**
 - Ejecución idéntica del backend escrito en Python 3.11/FastAPI en Windows, Linux y macOS.
 - Conmutación automática del almacén al motor SQLite3 local por archivo en ausencia de un gestor MySQL externo.
- **Métrica:**
 - Tiempo de instalación y configuración de la API < 5 minutos.
 - 100% de consistencia en el comportamiento del motor estático en cualquier SO.



11.5 Escenario de Usabilidad e Interfaz (Basado en RNF05)

- **Estímulo:** Visualización del panel interactivo de la extensión en diferentes pantallas y resoluciones.
- **Fuente:** Desarrollador interactivo utilizando el editor de código.
- **Artefacto:** Extensión de VS Code (Webview premium con estilos *glassmorphism*).
- **Entorno:** Editor con diferentes dimensiones físicas y temas de color activos (oscuro, claro).
- **Respuesta:**
 - Redimensionamiento dinámico y adaptativo de los componentes gráficos del panel.
 - Adopción en tiempo real de las variables CSS y colores HSL del tema visual activo del IDE.
- **Métrica:**
 - Tiempo de redibujado de la interfaz < 200ms.
 - 0 desalineaciones o fallos de visibilidad en resoluciones estándar.

11.6 Escenario de Extensibilidad (Basado en RNF06)

- **Estímulo:** Solicitud de incorporar una nueva regla o firma de seguridad para detectar inyección de plantillas Jinja2.
- **Fuente:** Desarrollador o analista de seguridad.
- **Artefacto:** Archivo core de firmas modulares (**scanner.py**).
- **Entorno:** Mantenimiento y actualización del motor de análisis.
- **Respuesta:**
 - Inserción del diccionario descriptivo de la regla en la lista constante **RULES** dentro de **scanner.py**.
 - No se requiere alterar la lógica interna de los métodos del motor de escaneo.
- **Métrica:**
 - Tiempo de incorporación y validación de la regla < 10 minutos.
 - 0 afectaciones en el funcionamiento de las reglas preexistentes (validado mediante pruebas unitarias).